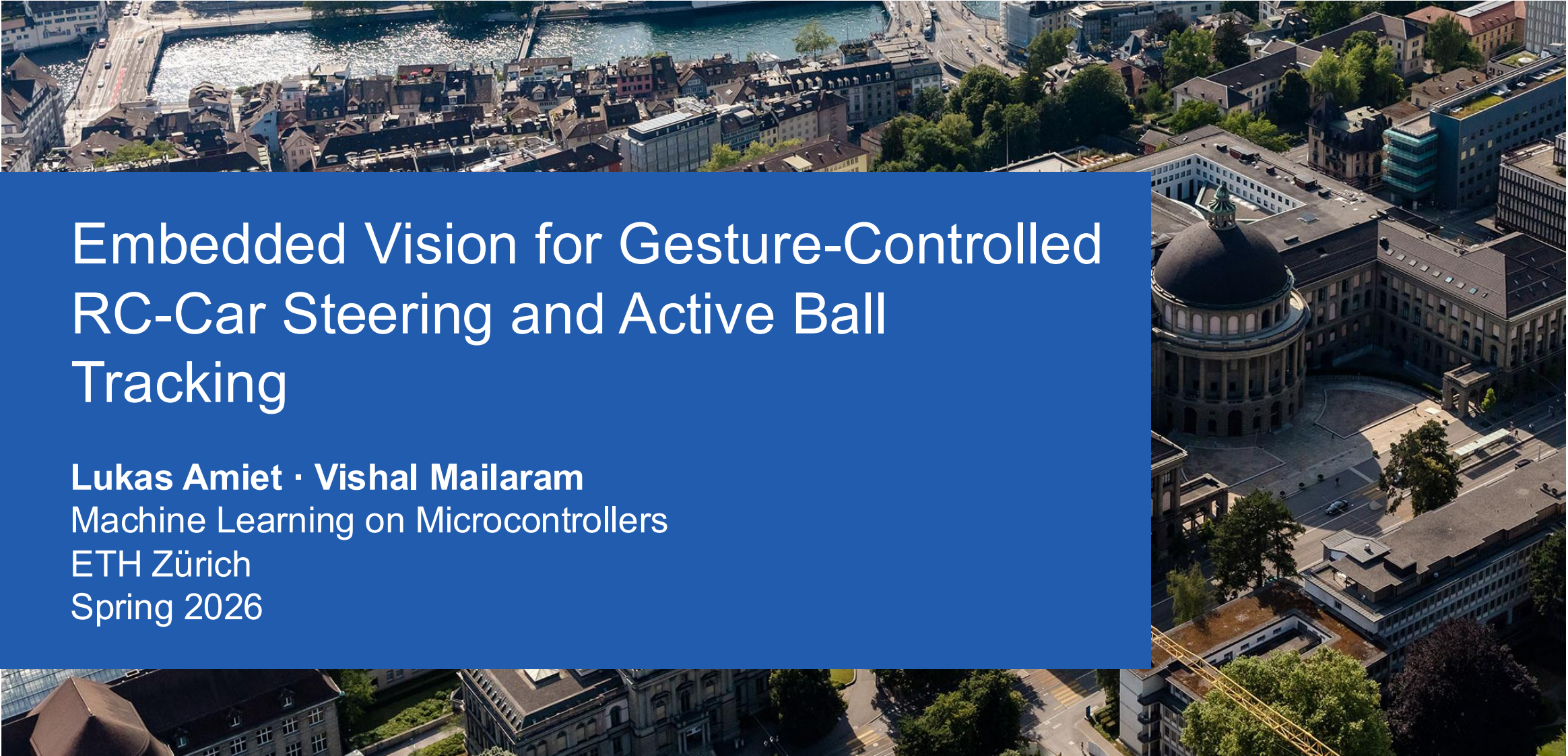




Embedded Vision for Gesture-Controlled RC-Car Steering and Active Ball Tracking

Lukas Amiet · Vishal Mailaram

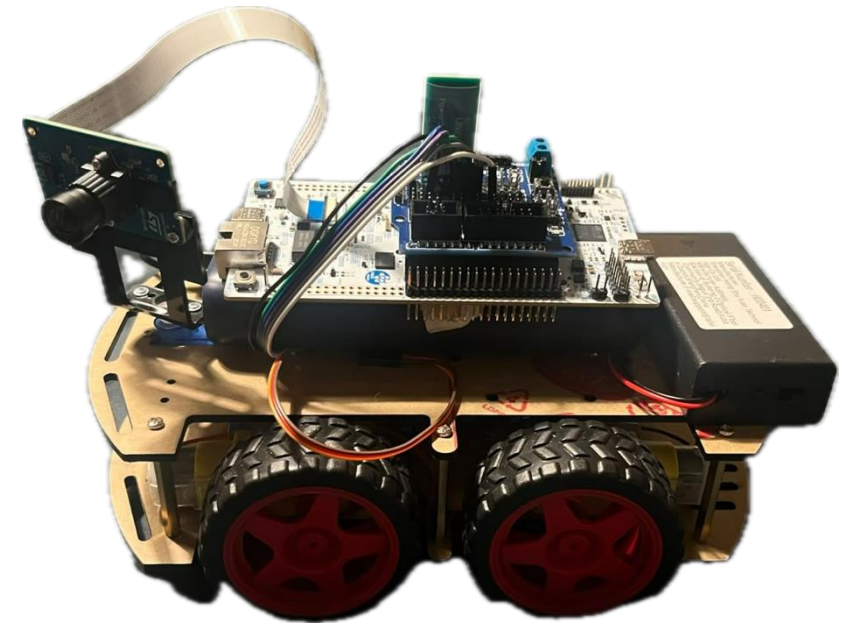
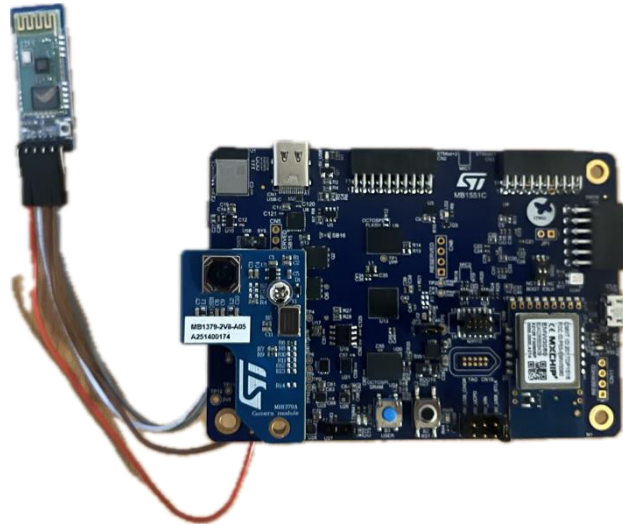
Machine Learning on Microcontrollers

ETH Zürich

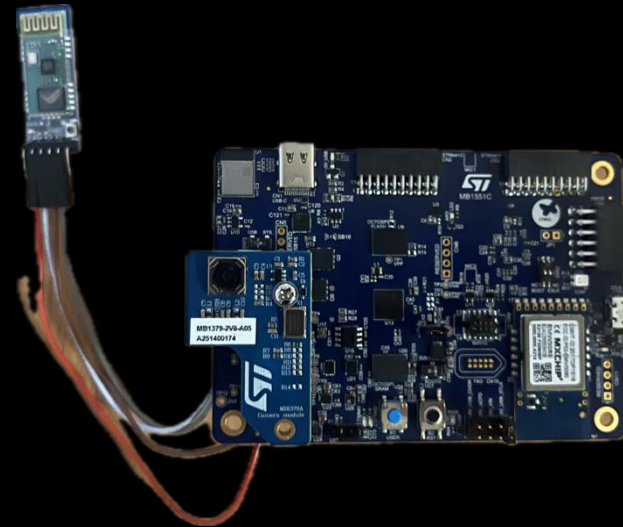
Spring 2026

Why This Project?

- Explore two ends of embedded ML : **STM32U5** and **STM32N6**
- Application
 - Gesture recognition for RC-car steering
 - Ball detection for active servo-camera lock-on



Hand Gesture Classification on STM32U5



Dataset — HaGRIDv2

- HAGRIDv2: Hand Gesture Dataset, with 34 classes
- Chose 6 intuitive classes for steering

Reference Classification on HaGRIDv2:

F1: 0.99; 58M params

<https://arxiv.org/pdf/2412.01508>



Rock (Drive Straight Forward)



Pinkie (Drive Right Forward)



Stop (Stop Driving)



One (Drive Left Forward)



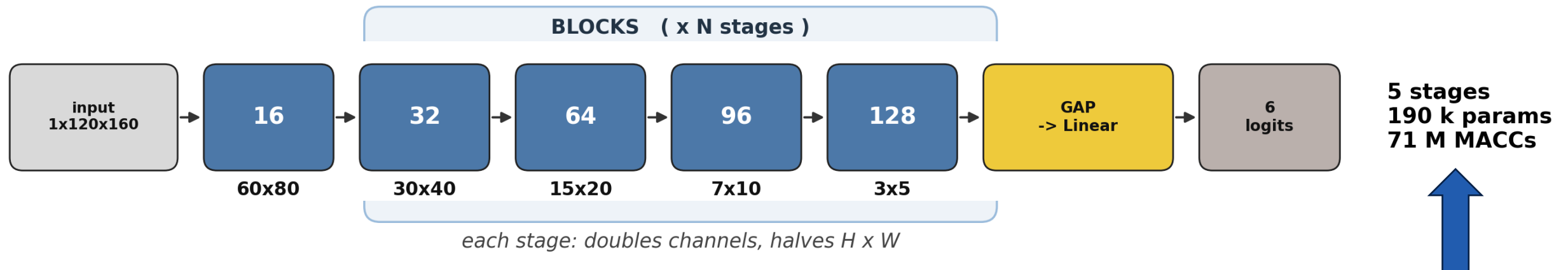
Fist (Drive Backwards)



Others (Non-Driving Gestures)

Architecture

- Hardware-aware from the start
 - Fewest labels that still steer a car
 - Only deployment-friendly ops (clean CubeMX/AI and .onnx export)
 - One simple repeatable block → easy scaling
- Loss: CrossEntropyLoss
- Some hyperparameters: Adam, CosineAnnealingLR, dropout



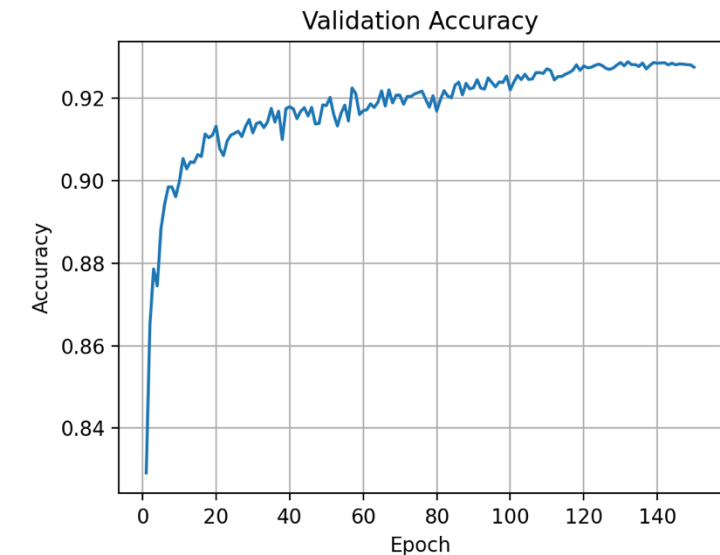
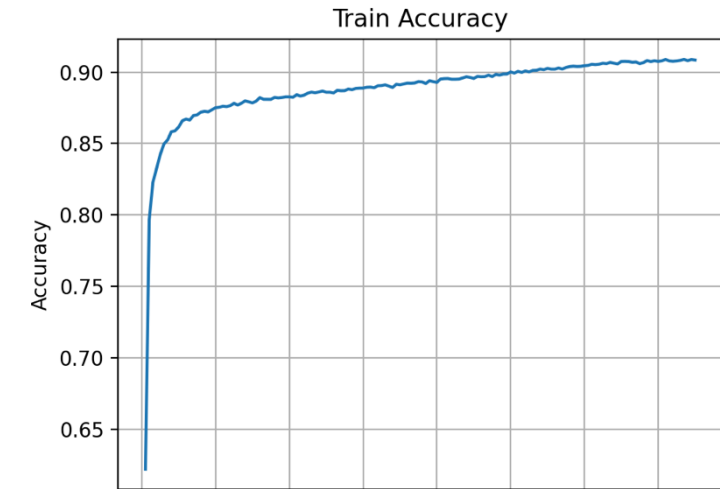
Hardware-Aware Optimization

(Classical) structured L1 norm channel Pruning

- Post-training pruning with fine-tuning to recover huge accuracy drop
- Reduced MACC and Params **around 10x**

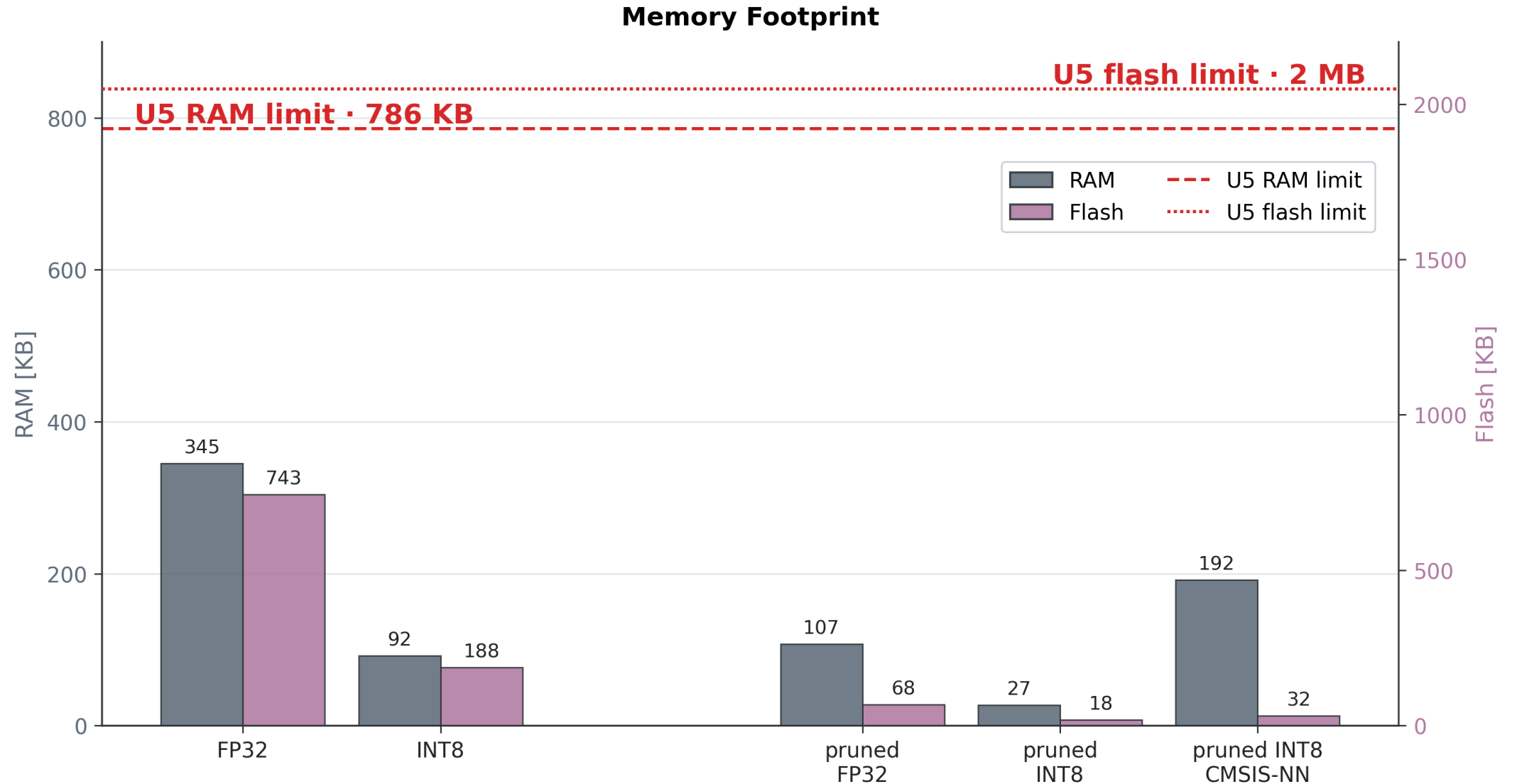
Quantization, INT8 PTQ

- Post-training quantization of weights and activations
- Calibrated on validation images
- Small till no accuracy drop



Fine-Tuning after pruning

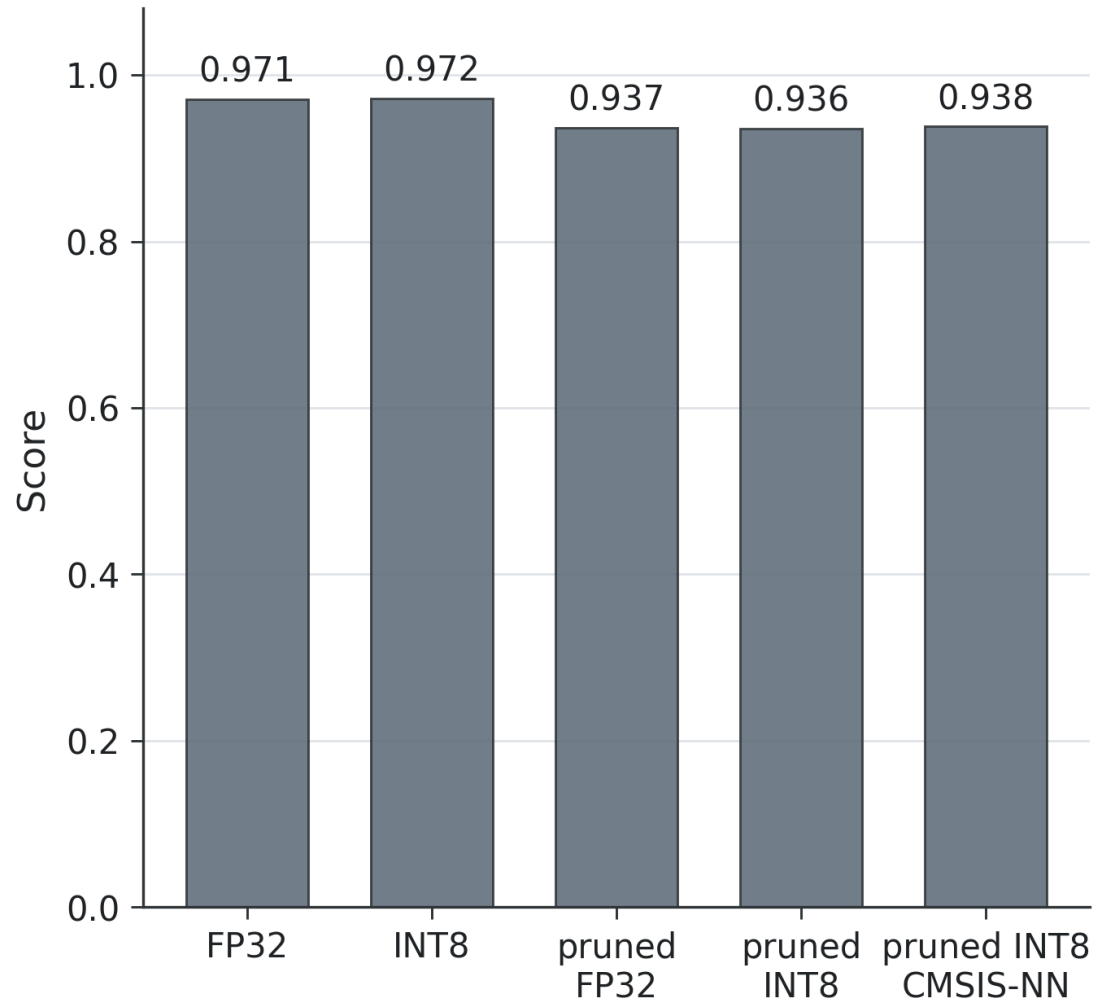
Results — Gesture Classifier (STM32U5 @ 160 MHz)



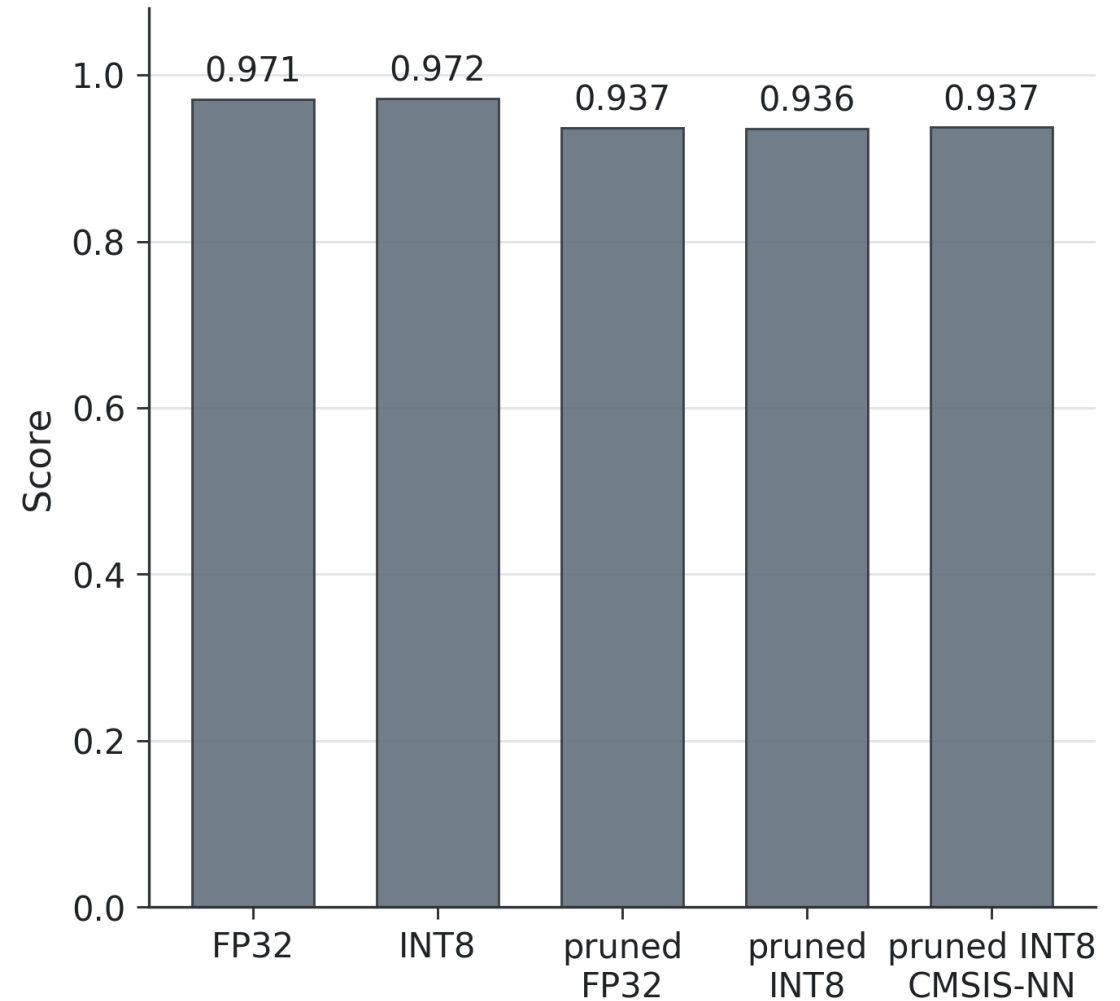
Results — Gesture Classifier (STM32U5 @ 160 MHz)

Model Metrics

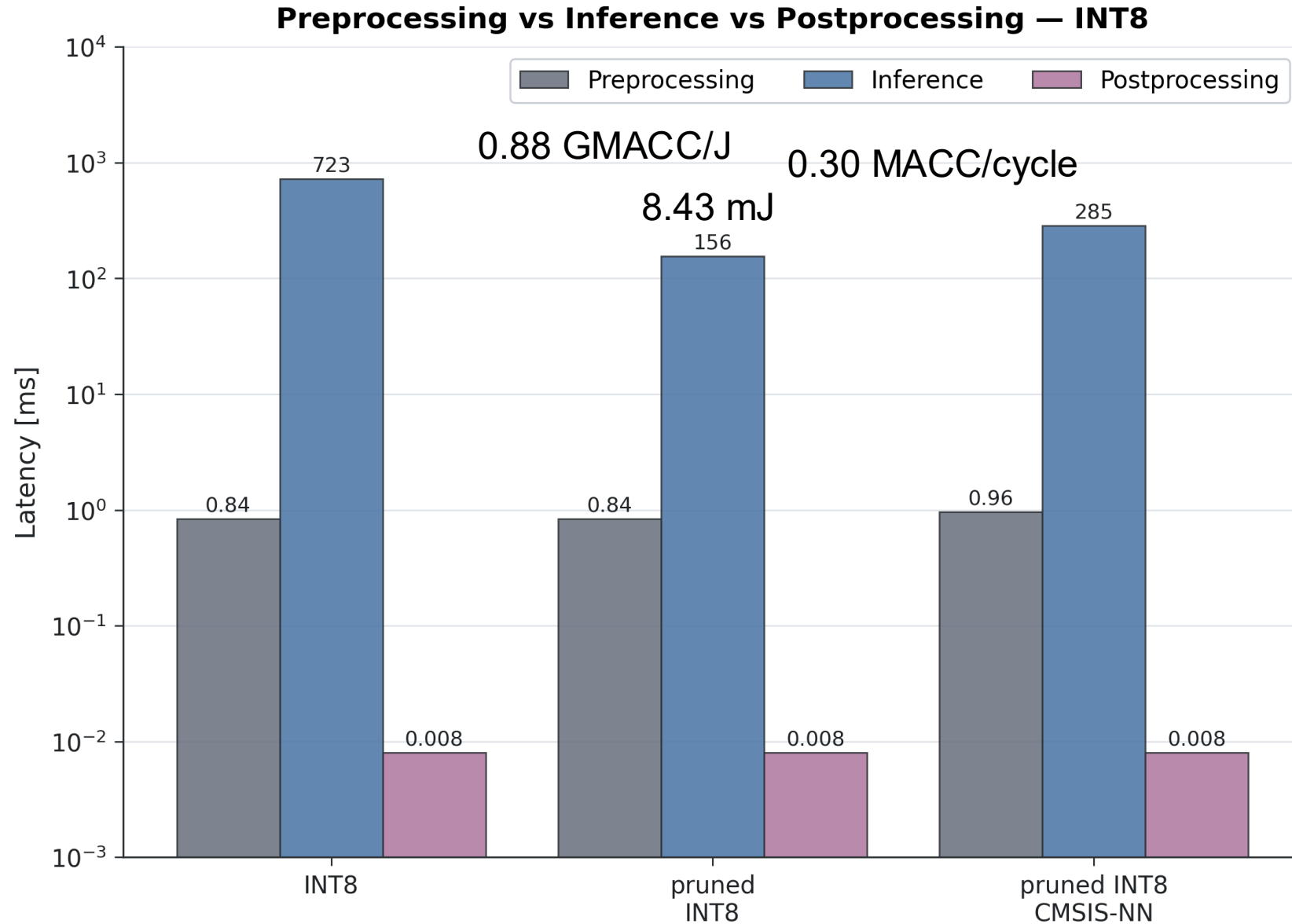
Accuracy



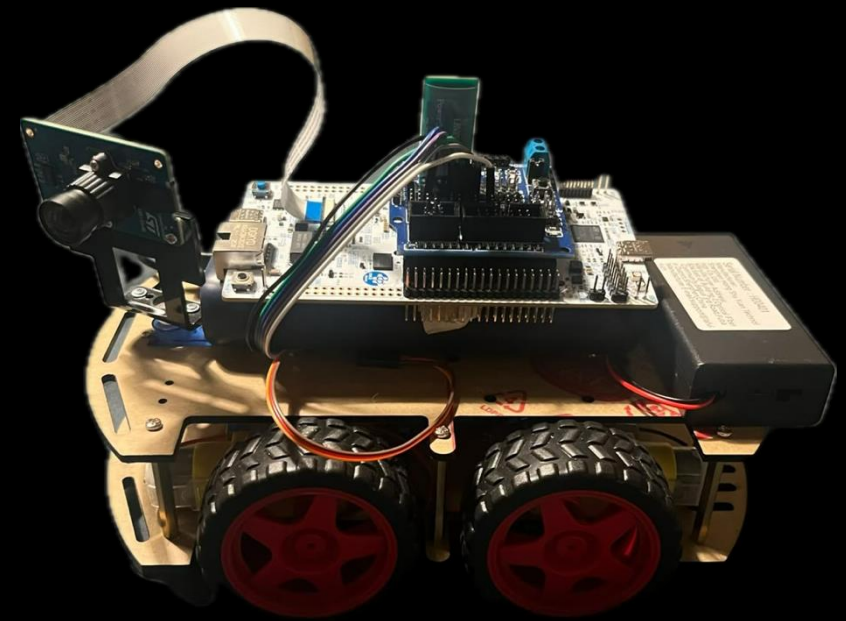
Macro F1



Results — Gesture Classifier (STM32U5 @ 160 MHz)



Ball Detection on STM32N6



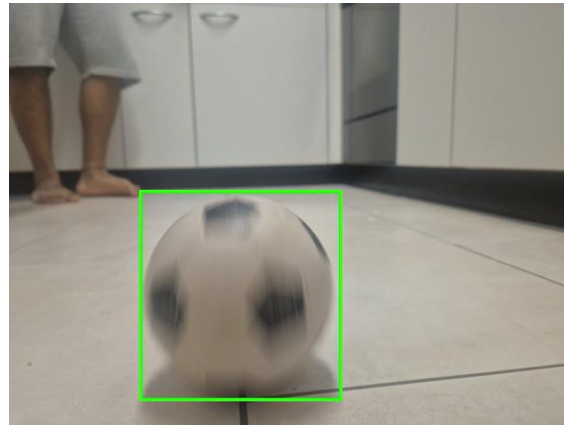
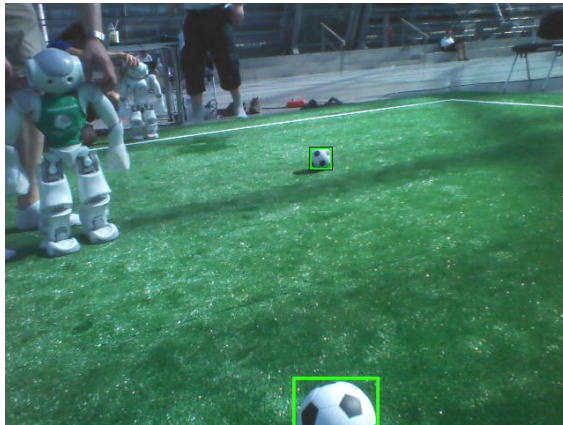
Dataset — RoboCup SPL + OurDataset

- **SPLBallDataset** → 2'400 images, always green floor
- **OurDataset** → 1'000 images, diverse floor
- **Combined:**
 - Diverse floor / lighting / viewpoint / motion blur
 - Ball size from 6px - 300px
 - Augmented in dataloader (hflip, color-jitter, blur)

Reference Model: STYOLO-Nano (2026)

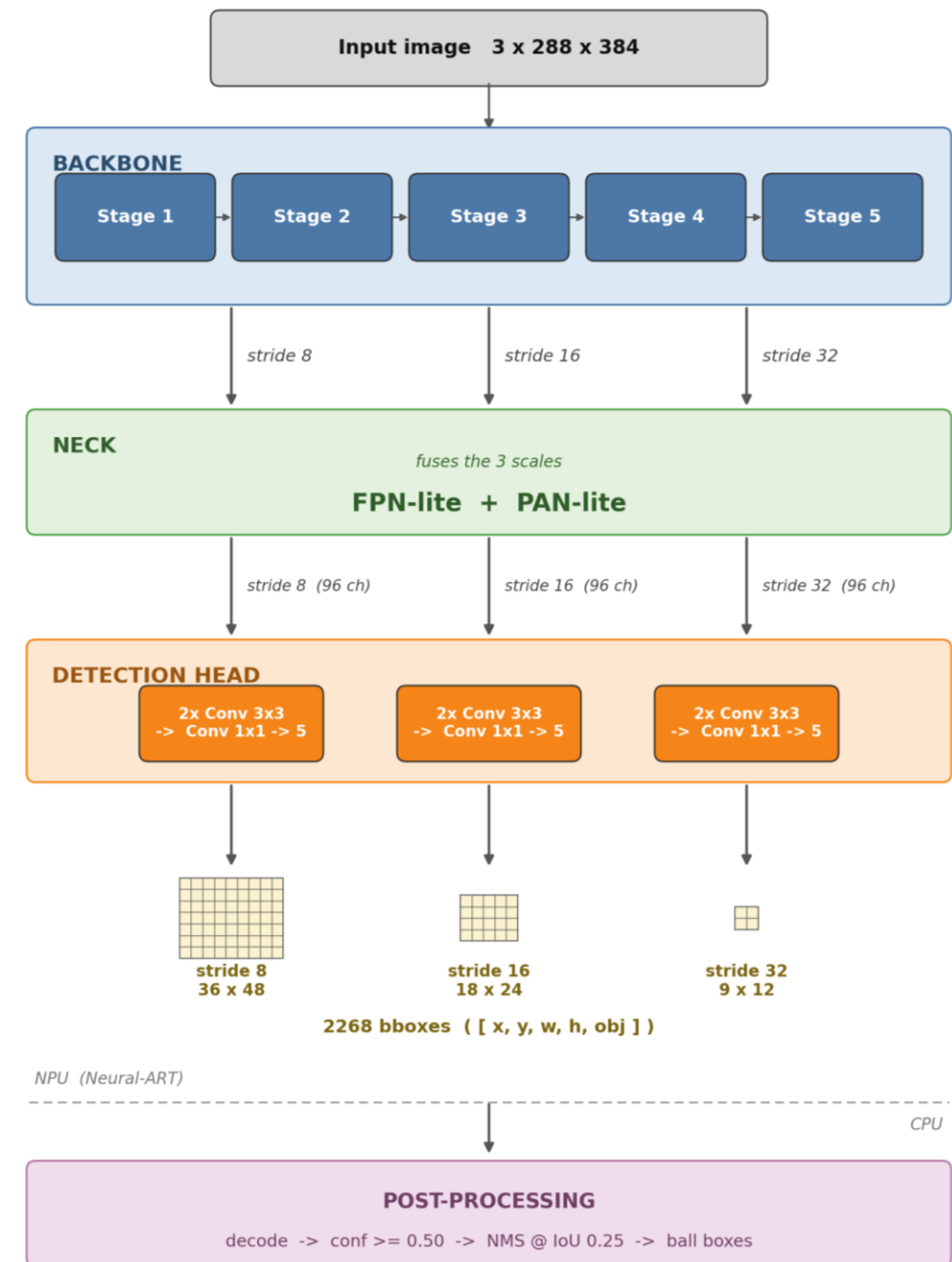
41.4 ms, 80 classes STM32N6

<https://arxiv.org/pdf/2601.05364>

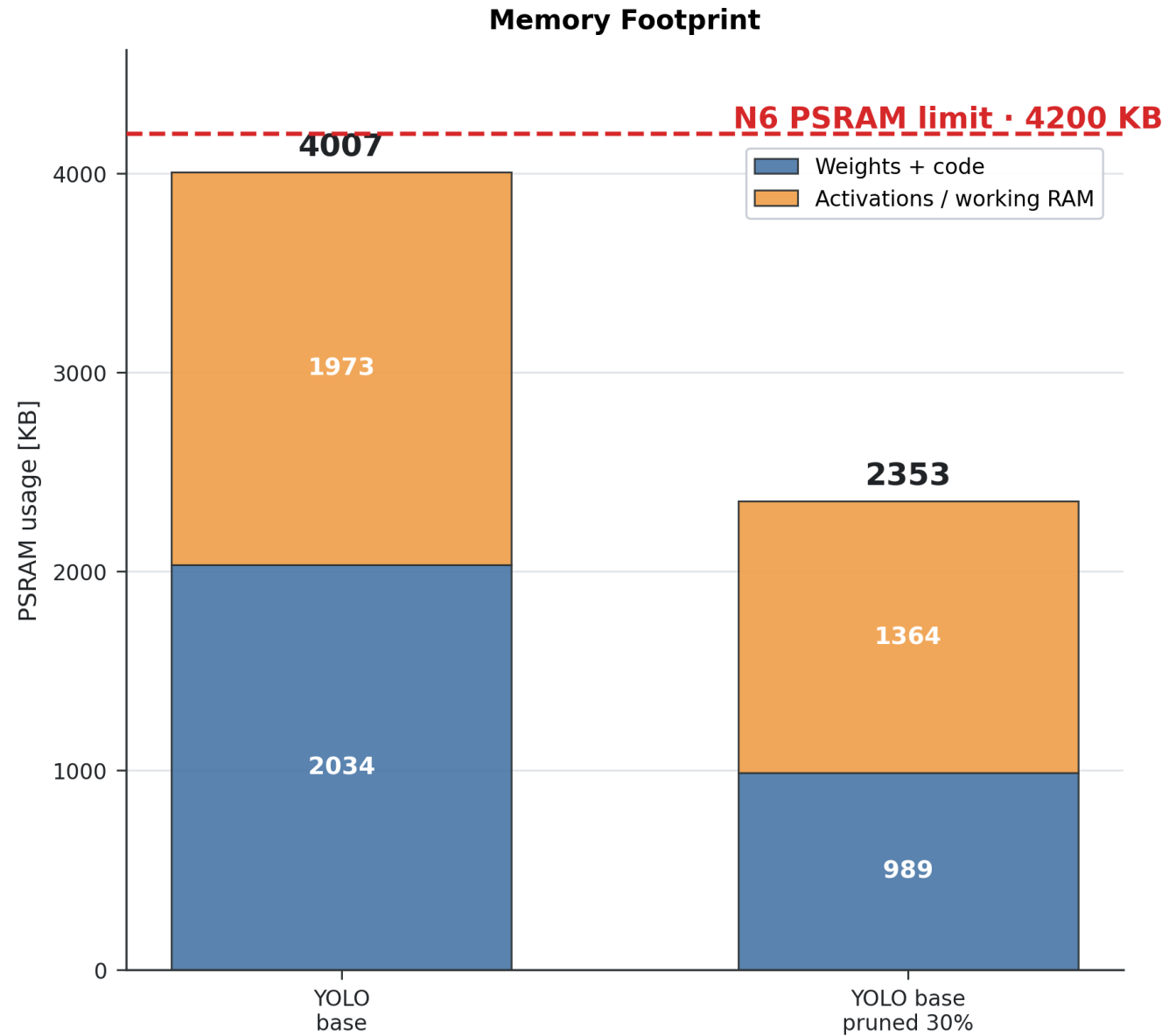


Architecture — Model & optimization

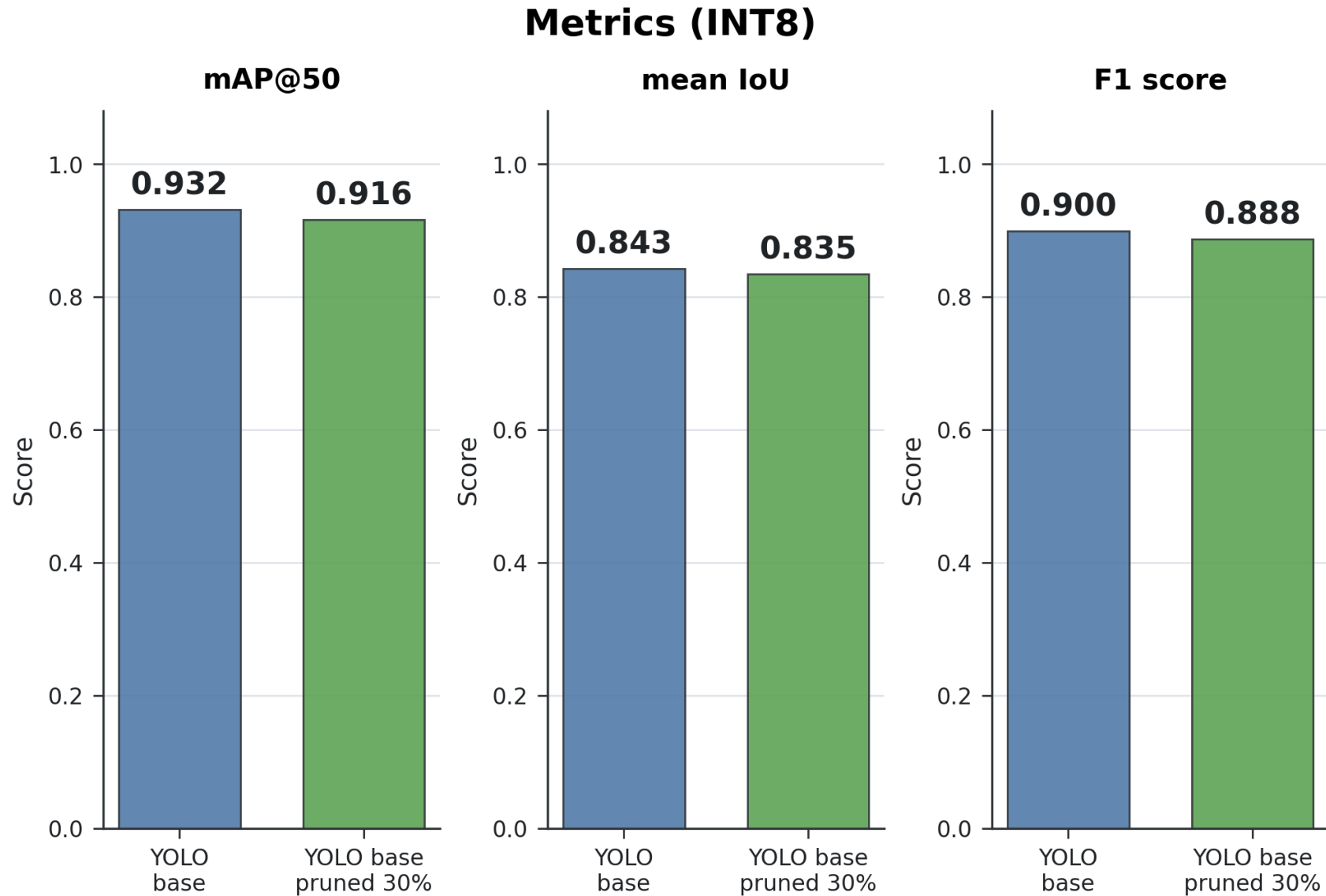
- Model
 - NPU-friendly ops only
 - YOLO-style, anchor-free, 3 scales (stride 8/16/32)
 - Decode + NMS kept off-graph
 - Objectness-only head: 5 outputs/cell (4 box + 1 obj)
 - 2.086 M Params, 1.285 GMACCs
- Optimizations
 - SimOTA label assignment
 - Loss = Focal obj. loss + DIoU box loss
 - FP32 8.1MB → INT8 4.0MB
 - Pruning -30% L1 channel with fine tuning
→ 1.01M params, 0.62 GMACCs



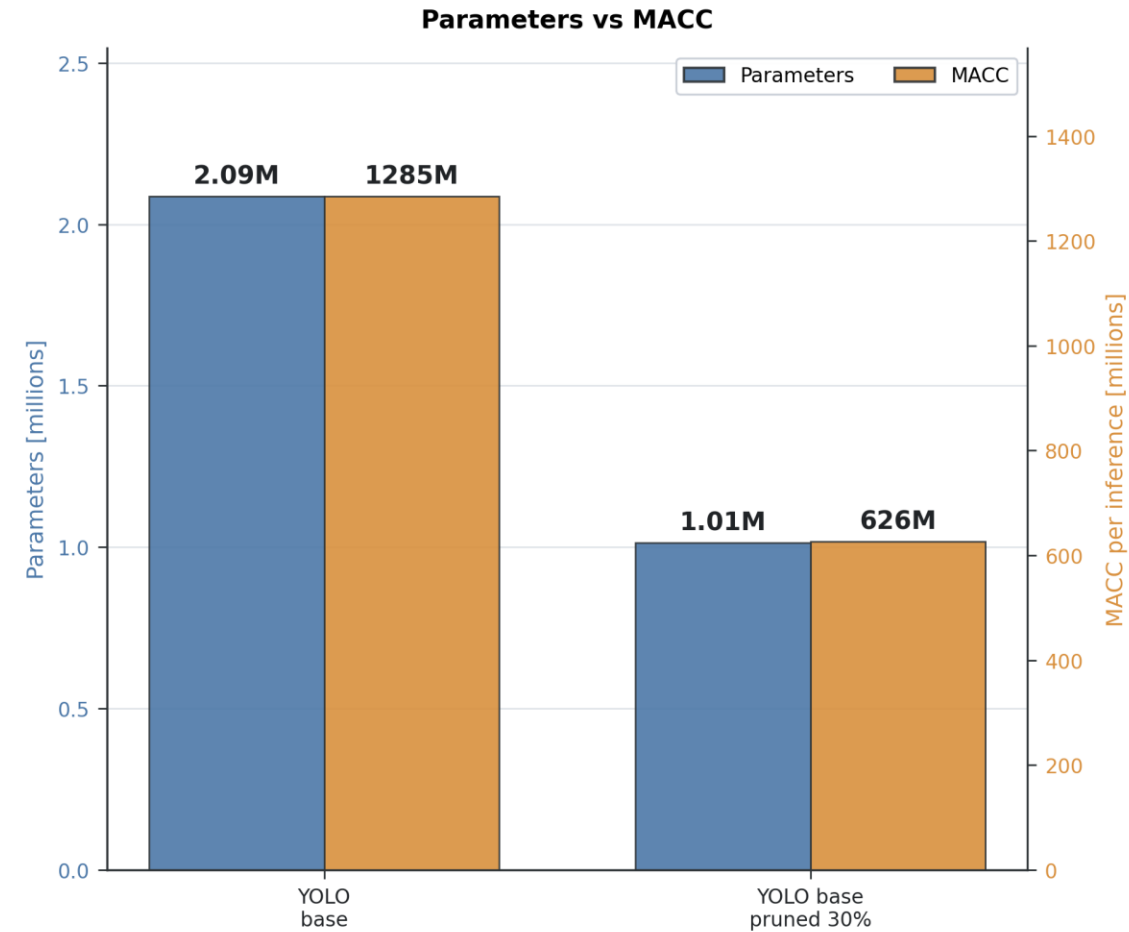
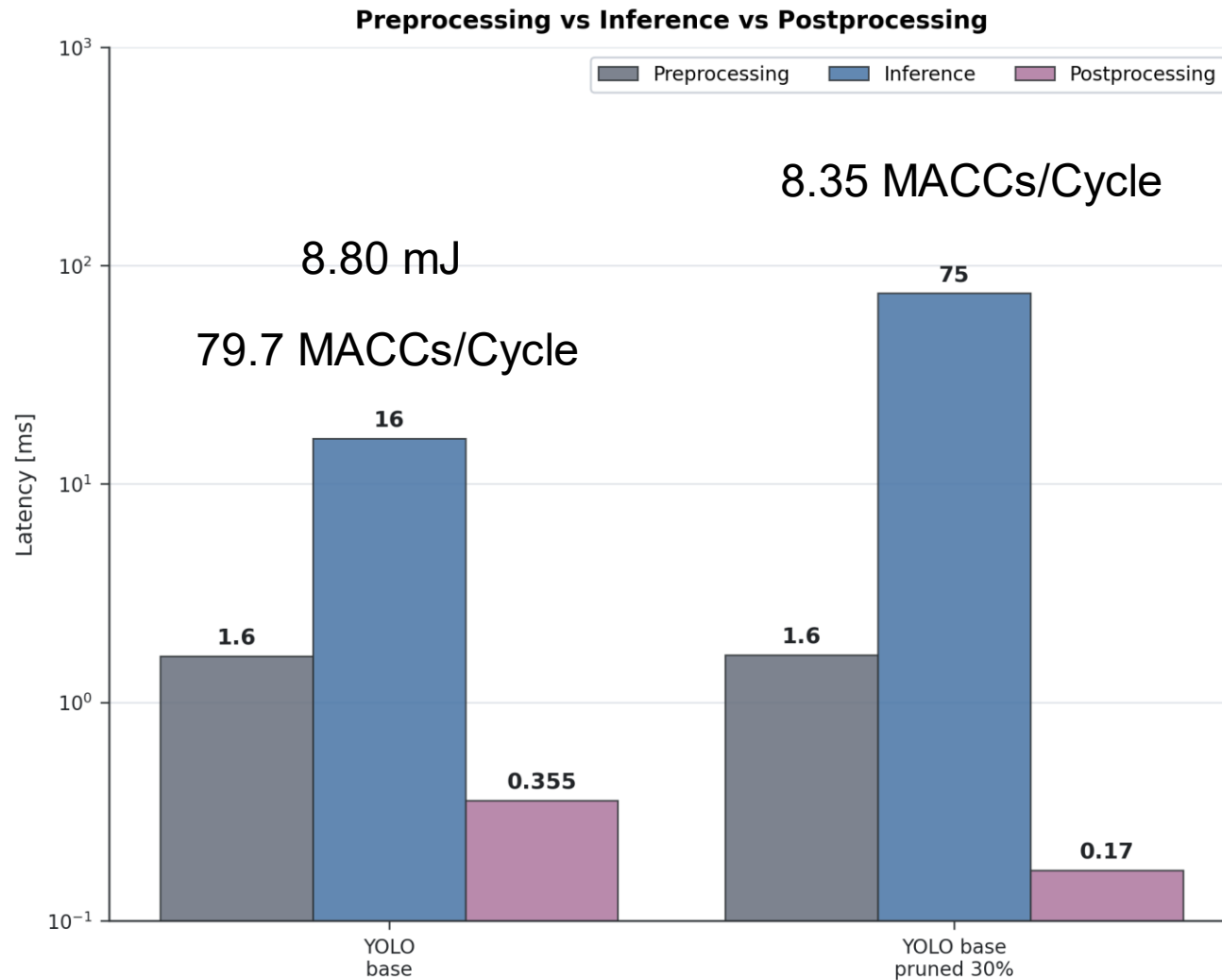
Results — Ball Detector (STM32N6)



Results — Ball Detector (STM32N6)



Results — Ball Detector (STM32N6 — NPU 1GHz, CPU 800MHz)



Demo

Meet
PAUL

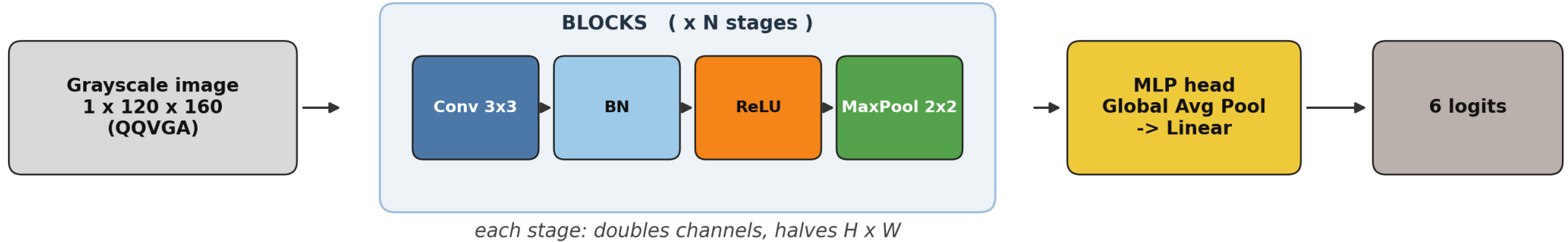


<https://youtu.be/WXF68EdYNcE>

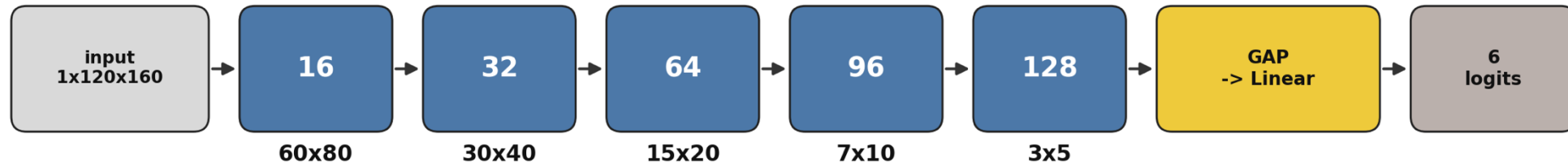
Appendix

Architecture — SmallNet vs BigNet

Overarching architecture:

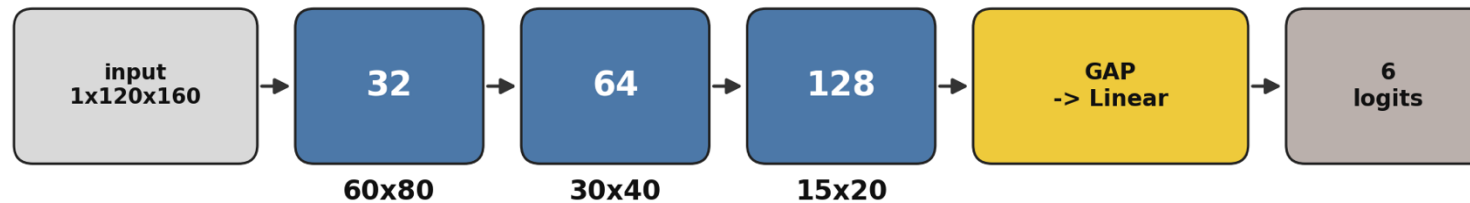


BigNet



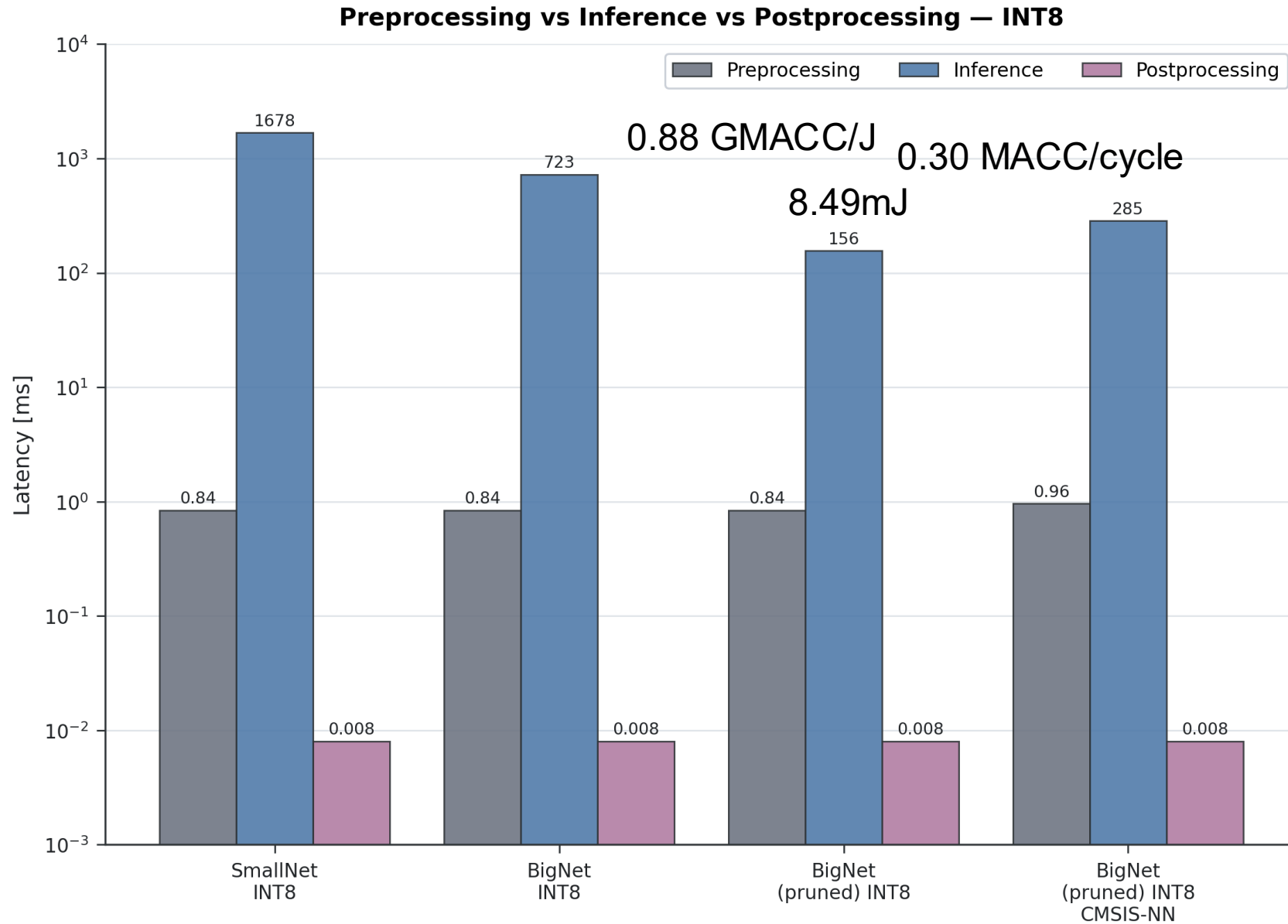
5 stages
190 k params
71 M MACCs

SmallNet

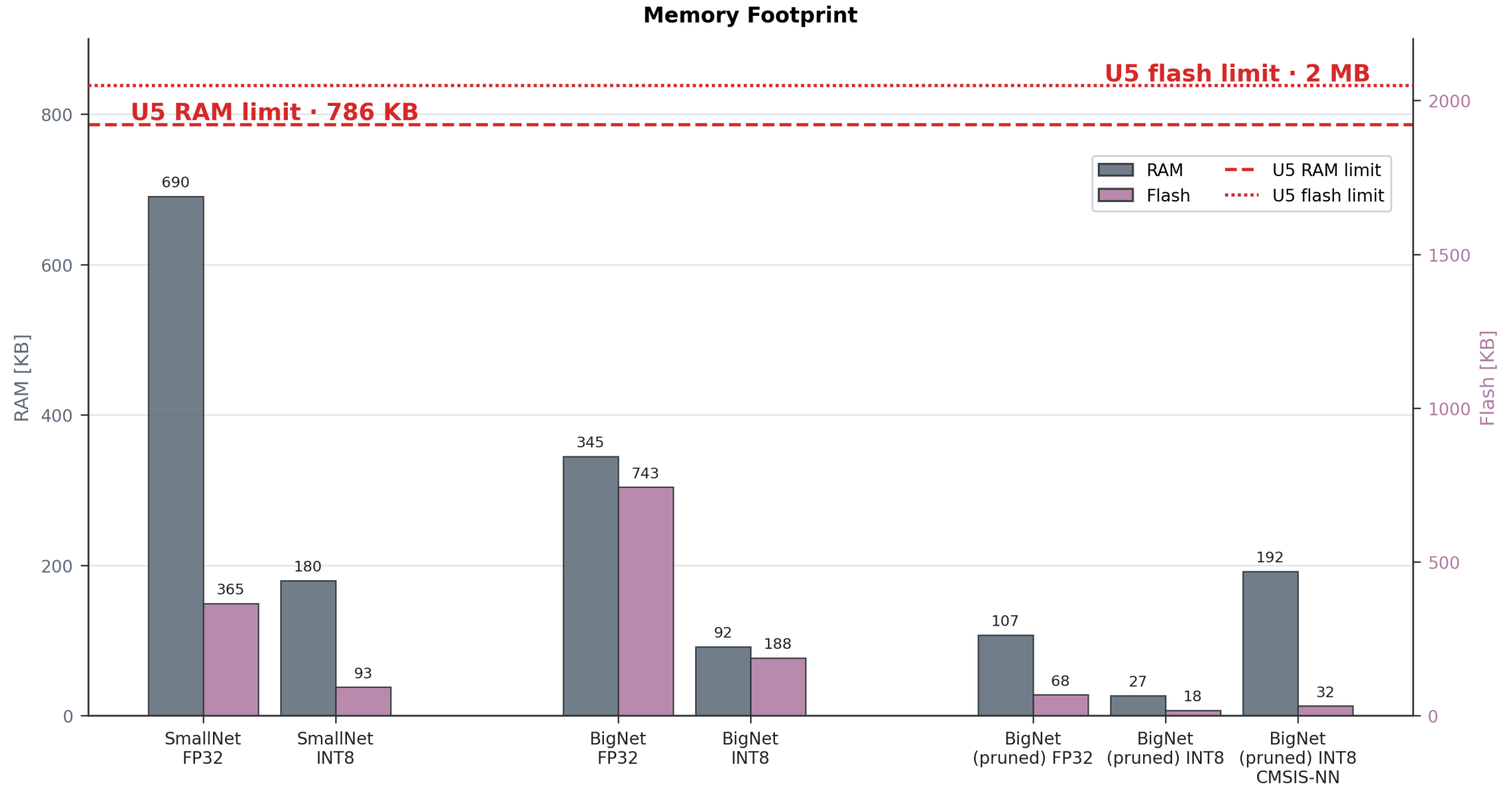


3 stages
94 k params
182 M MACCs

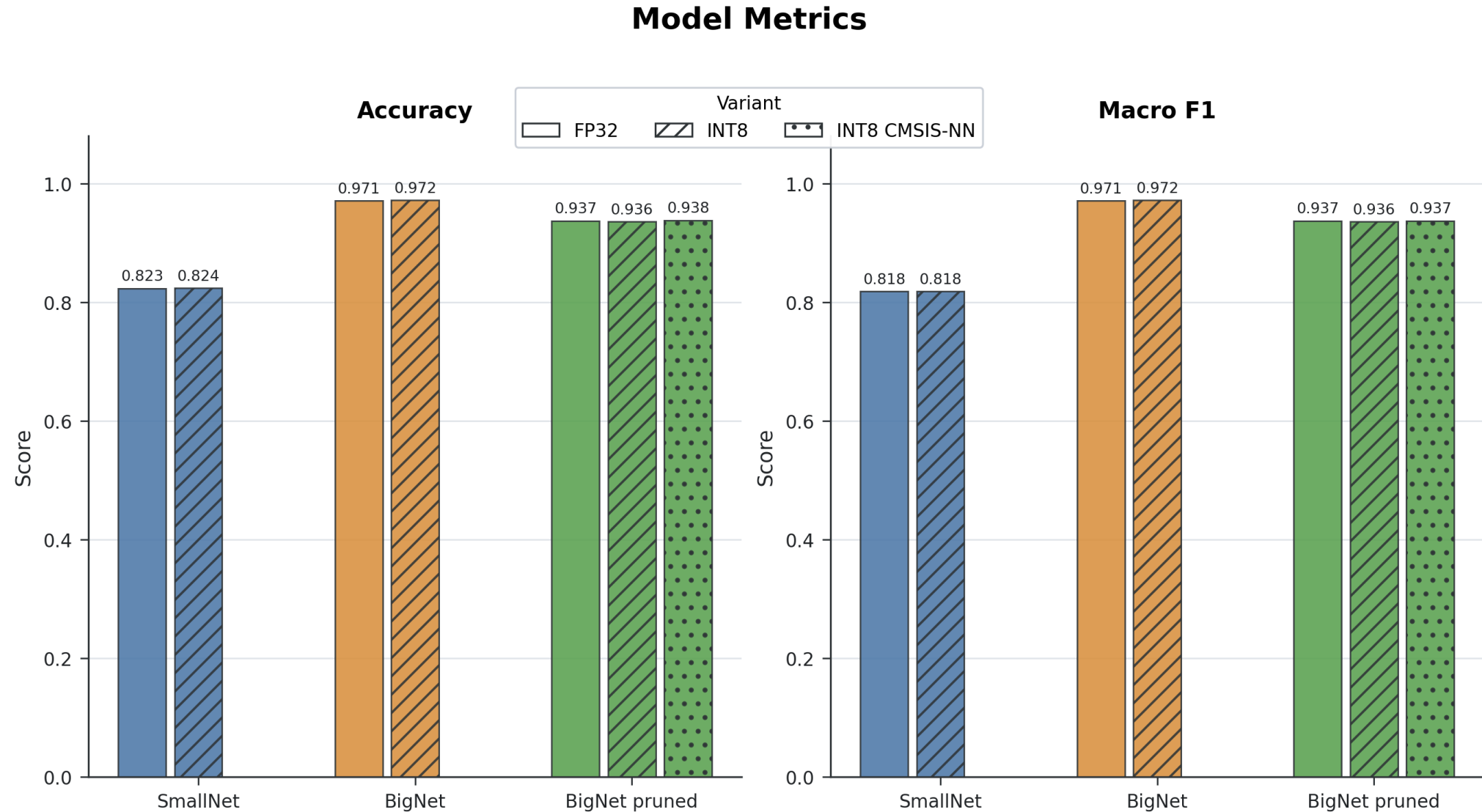
Results — Gesture Classifier (STM32U5 @ 160 MHz)



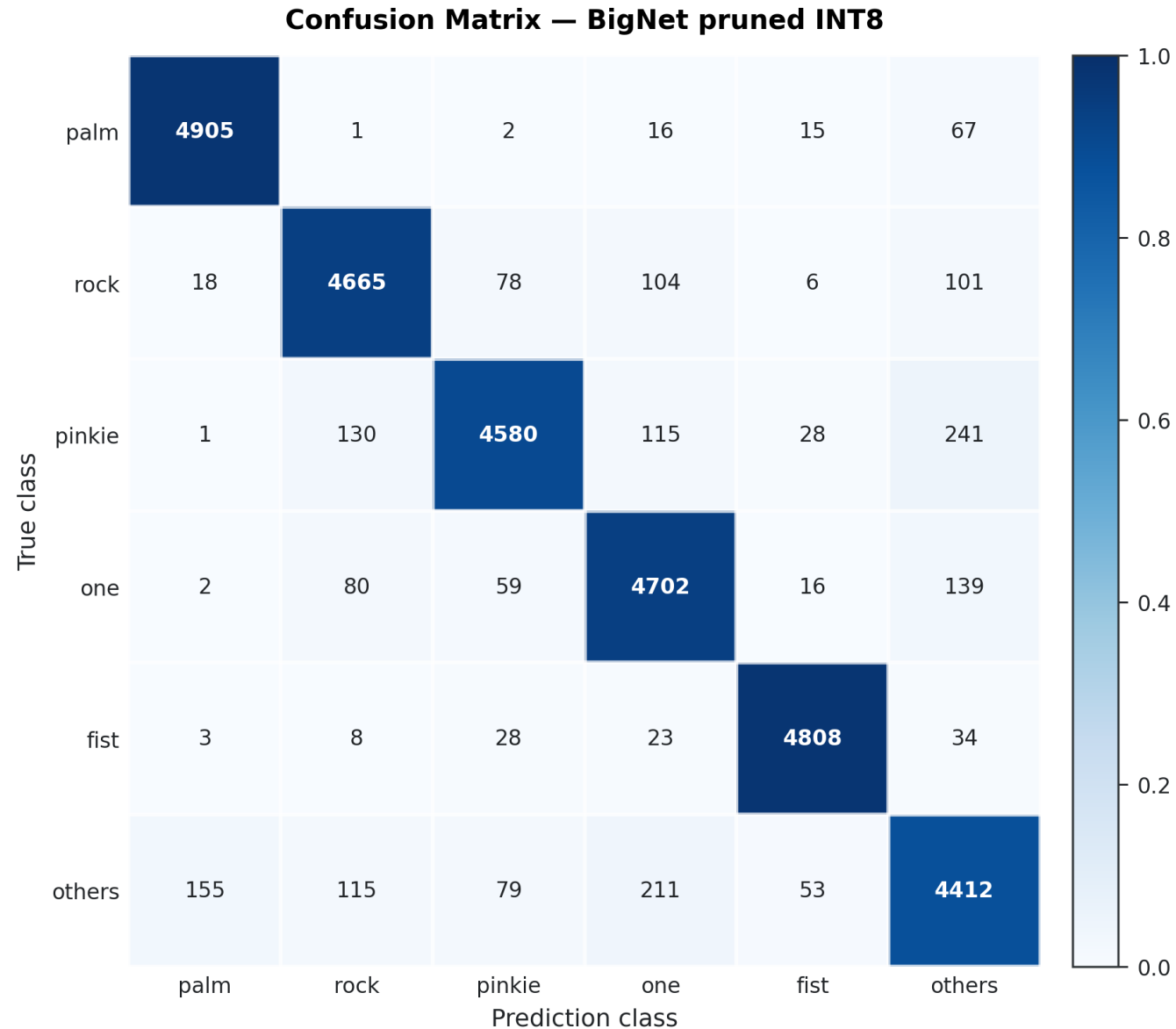
Results — Gesture Classifier (STM32U5 @ 160 MHz)



Results — Gesture Classifier (STM32U5 @ 160 MHz)



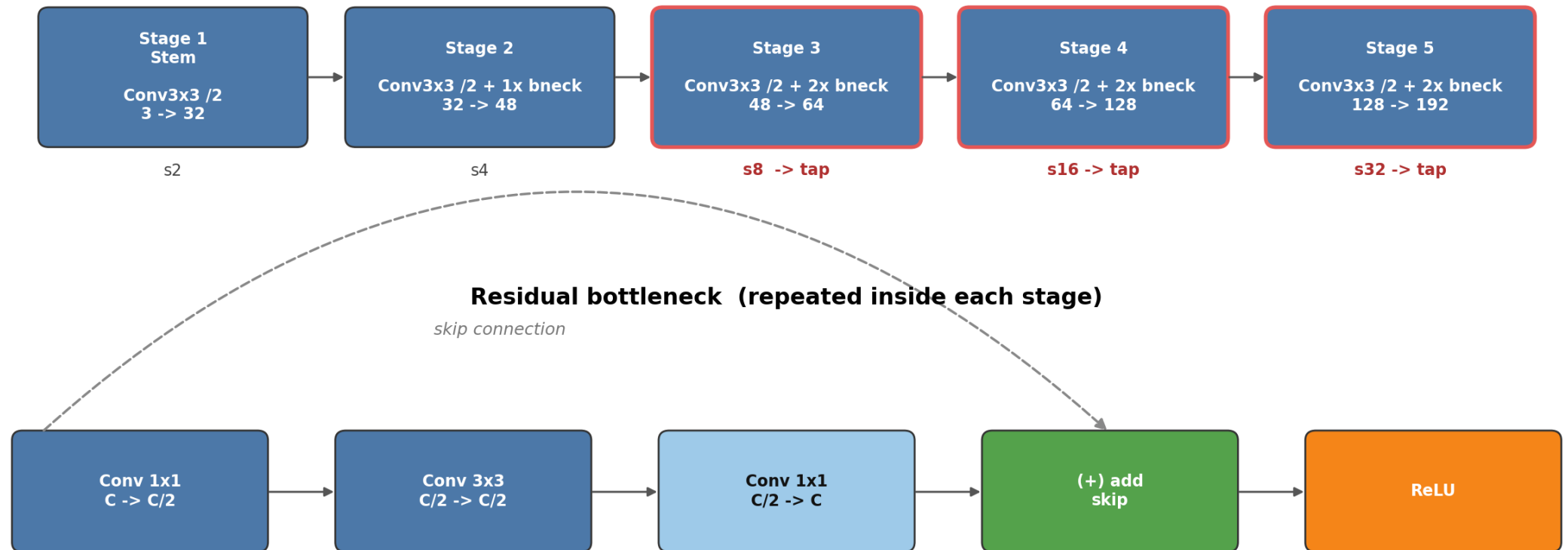
Results — Gesture Classifier (STM32U5 @ 160 MHz)



Ball Detection — Backbone

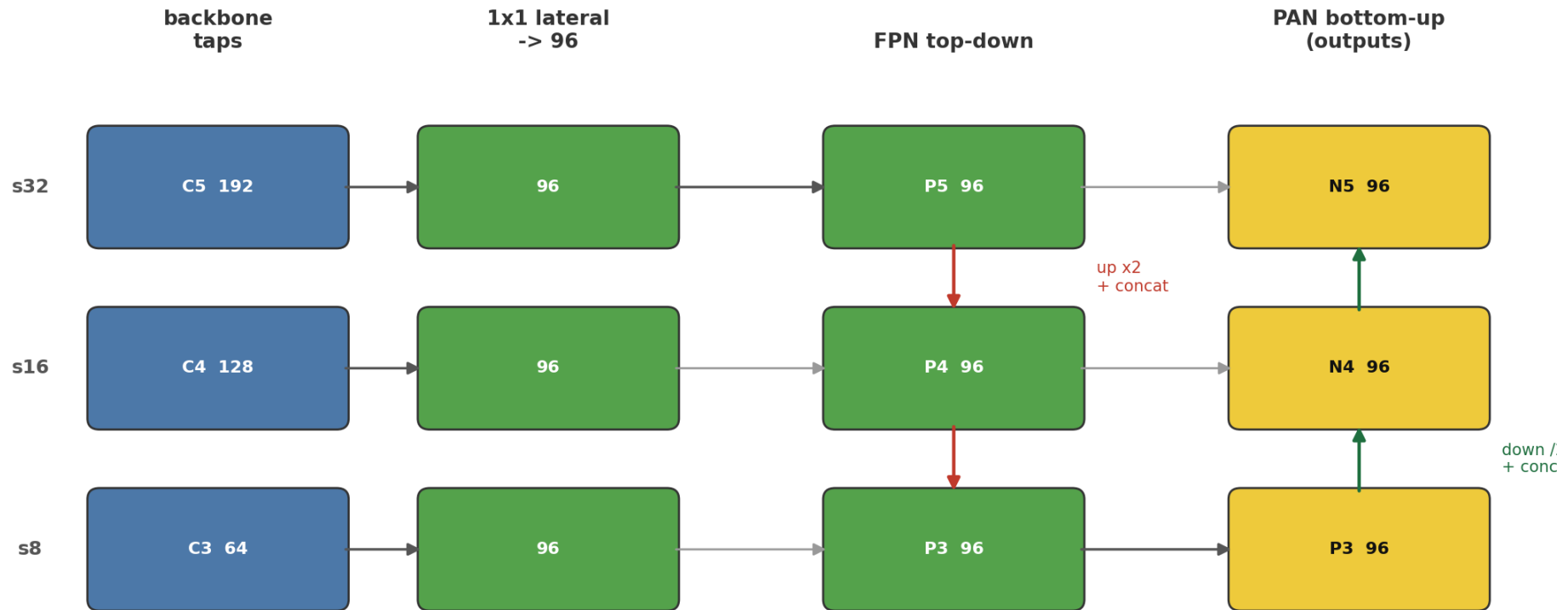
Backbone — full stack & residual bottleneck

spatial size halves each stage · channels grow · Stages 3/4/5 feed the neck



Ball Detection — Neck

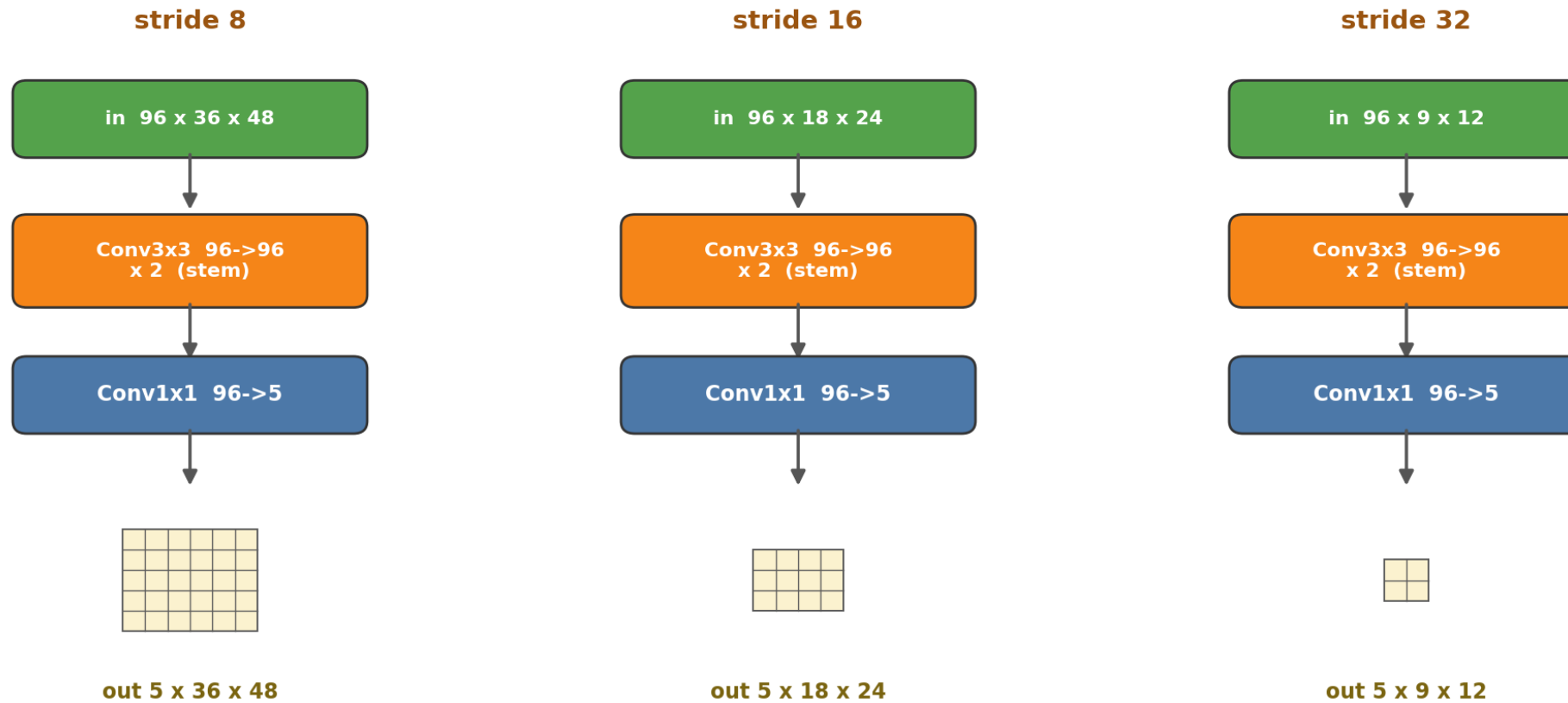
Neck — FPN-lite (top-down) + PAN-lite (bottom-up)



all scales unified to 96 channels -> P3 / N4 / N5 go to the detection head

Ball Detection — Head

Detection head — one per scale (shared structure), anchor-free, single class



5 channels per cell = [tx, ty, tw, th, objectness] · 2 268 cells total

Ball Detection — Loss

- **Loss = 1 * L_obj + 5 * L_box**
- **Objectness L_obj — Focal BCE** (every cell)
 - Ca 99.9% of cells are empty → imbalance
 - Plain BCE → model predicts "nothing everywhere"
- **Box — DIoU** (positive cells only)
 - IoU: scale-invariant, optimizes the eval metric (mAP@IoU)
 - Plain IoU = flat gradient when no overlap (no direction)
 - DIoU's center term pulls the box toward the ball even at zero overlap → faster, stabler

$$L_{BCE} = -\frac{1}{n} \sum_{i=1}^n (Y_i \cdot \log \hat{Y}_i + (1 - Y_i) \cdot \log (1 - \hat{Y}_i))$$

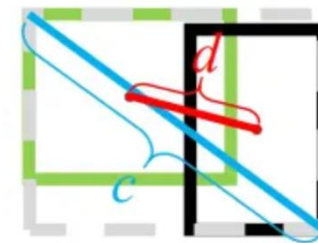


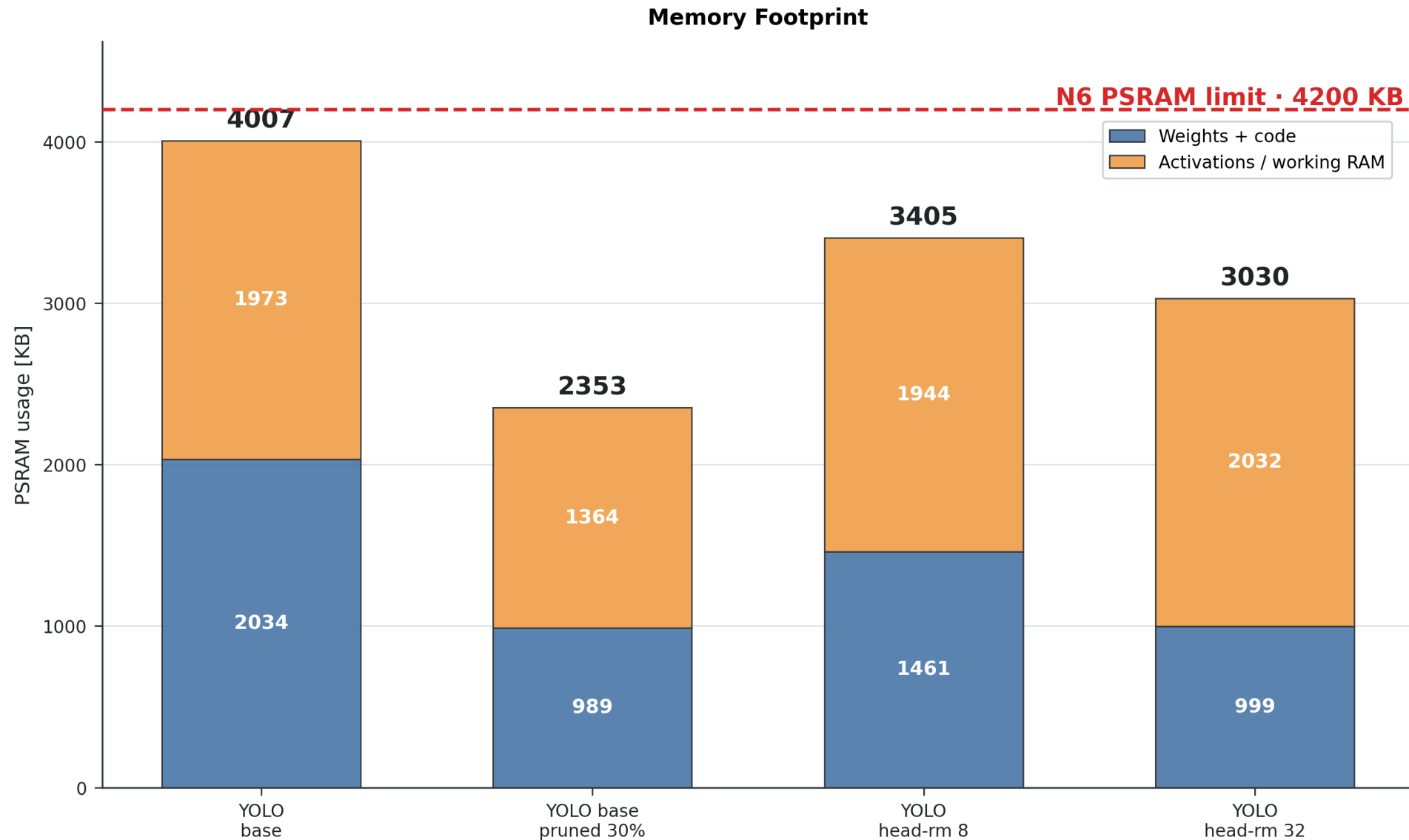
Fig. 7: Penalty explained; Source: [\[Link\]](#)

$$\mathcal{L}_{DIoU} = 1 - IoU + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2}$$

Ball Detection — SimOTA; Dynamic Label Assignment

- decide which grid cells become positives for each ball
- **Before — Center assigner (static)**
 - Ball → the single cell its center lands in
 - blind to predictions: forces a fixed cell even if a neighbor predicts a better box
- **Now — SimOTA (dynamic, cost-based)**
 - Candidate pool: all cells inside the ball region
 - **Cost per cell** = $3 * (-\log \text{IoU}) + 1 * \text{obj-BCE} + 0.5 * \text{center-dist}$ → scores the cell's prediction
 - Lower cost = better match
 - Dynamic-k: compute k from that GT's top IoUs: take the k lowest-cost cells → these become positives for that GT. → more positive cells, more gradient signal
 - Conflict: if two balls claim a cell, lowest cost wins

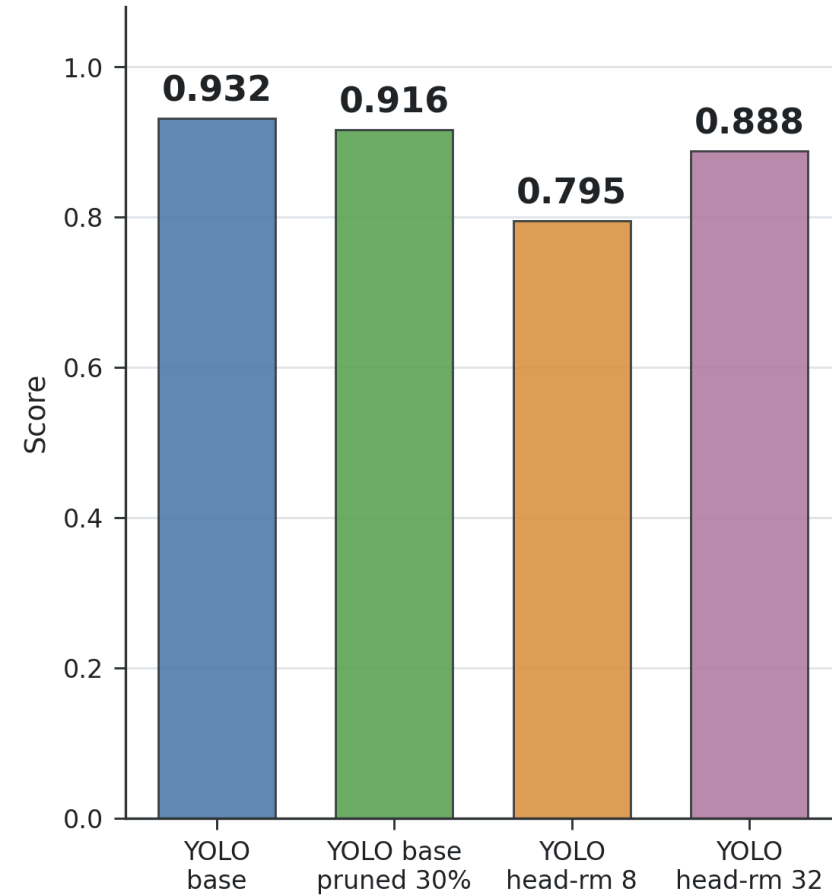
Results — Ball Detector (STM32N6)



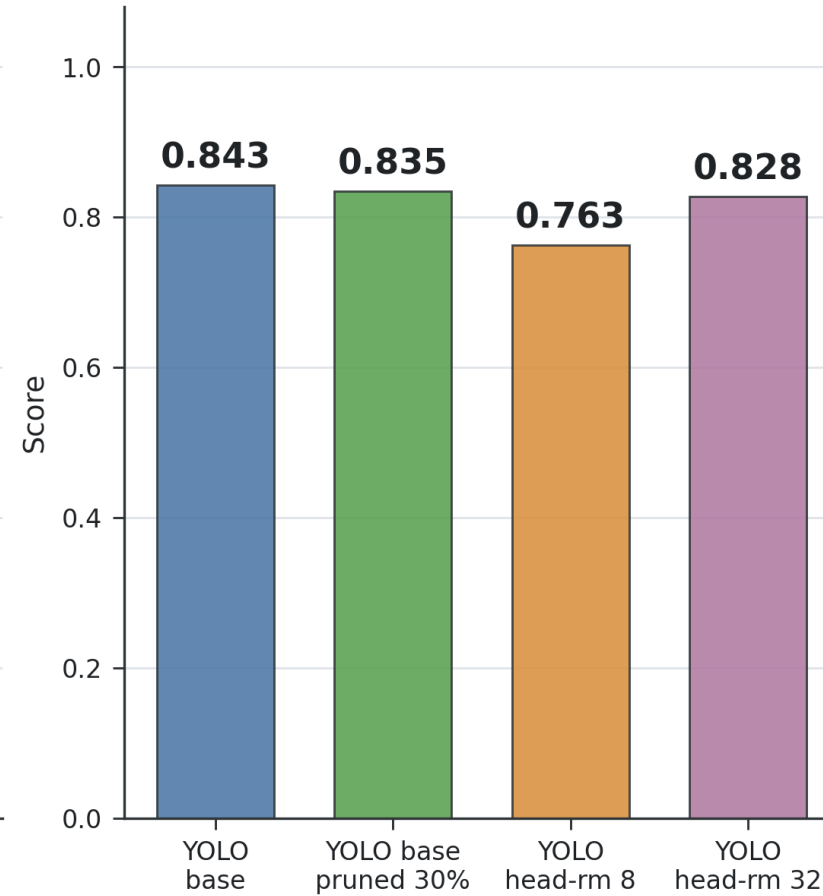
Results — Ball Detector (STM32N6)

Metrics (INT8)

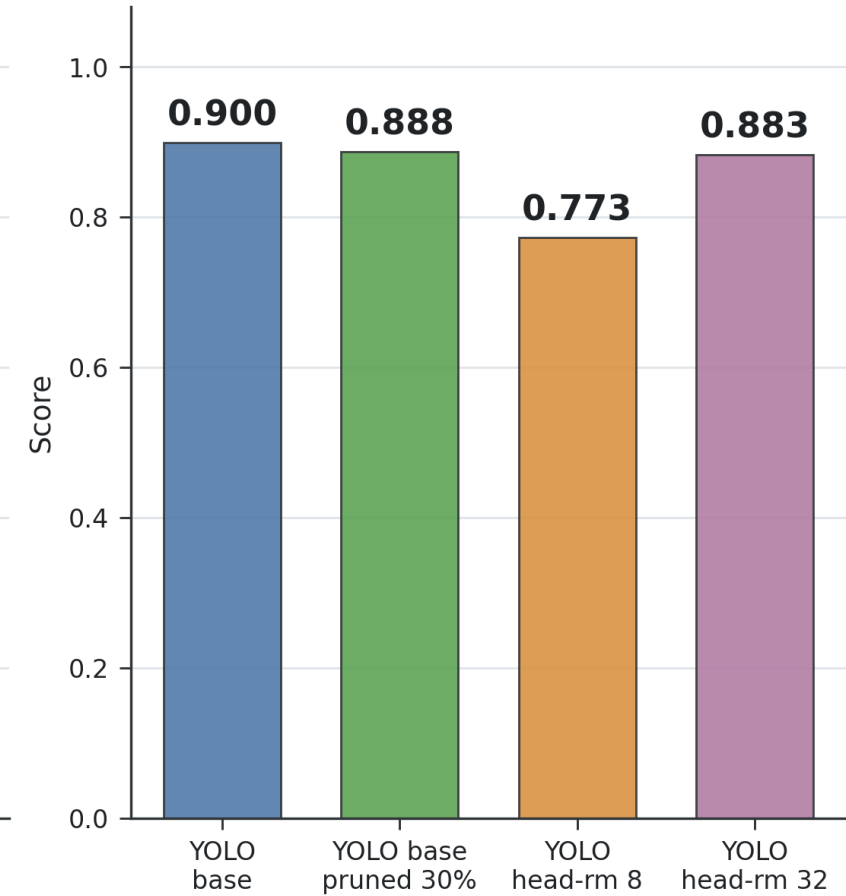
mAP@50



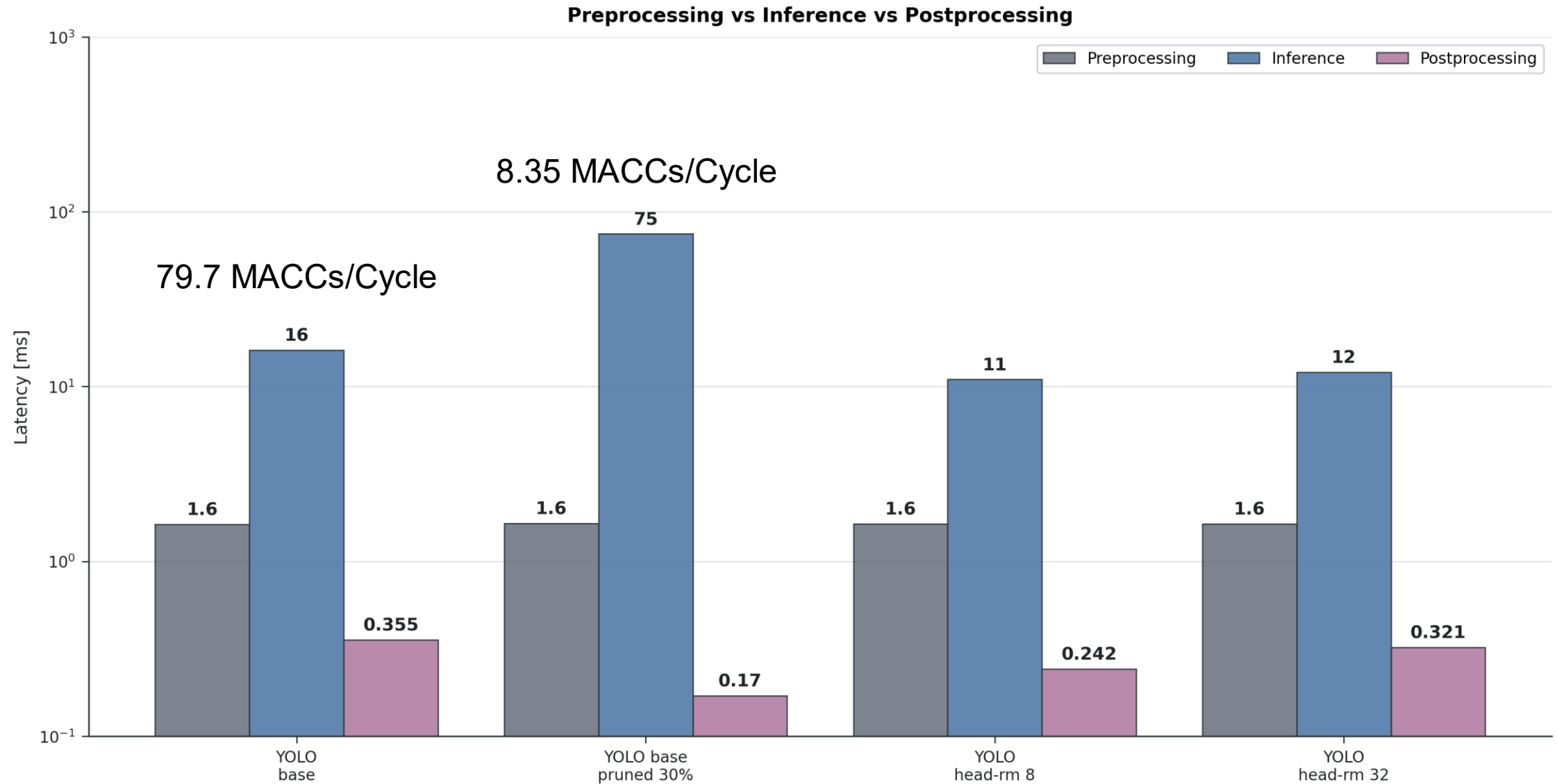
mean IoU



F1 score



Results — Ball Detector (STM32N6 — NPU 1GHz, CPU 800MHz)



GitHub

<https://github.com/amietlukas/embedded-vision-rc-car>